



BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Prompt Tuning

A Study on Task-Specific Prompt Learning

Sk Arib Rajin Shahan Sadman Zaman Fatin Nehal Zubaeer

2305068

2305075

2305076

CSE 200: Technical Writing and Presentation

September 23, 2026

Abstract

Large pretrained language models are usually adapted to a new task by fine-tuning all of their weights, which leaves one full copy of the model for every task. Prompt tuning avoids this cost. It keeps the model frozen and learns only a short sequence of continuous vectors, called a *soft prompt*, that is placed in front of the input. In this report we explain how prompt tuning works and when it is worth using. We define the method mathematically and show that a prompt of m tokens for a model with embedding dimension d adds only $m \cdot d$ trained parameters per task. For the 11-billion-parameter T5-XXL model with a 100-token prompt this comes to 409,600 parameters, or 0.0037% of the model. We then follow one gradient step on a small sentiment task by hand to show how the loss is computed and why only the prompt changes. The experiments of Lester et al. show that prompt tuning catches up with full fine-tuning as models grow, that it transfers better to new domains (by up to 12.5 F1 points), and that it makes ensembles cheap. We also compare the method with related parameter-efficient techniques, suggest when to prefer it over fine-tuning, and discuss the problems that remain open, namely slow training, sensitivity to initialization, weak results on small models, and prompts that people cannot read.

Keywords: prompt tuning; soft prompts; parameter-efficient fine-tuning; large language models; transfer learning.

Contents

1	Introduction	3
1.1	Objectives	3
1.2	Scope and Organization	4
2	Background	4
2.1	Language Models and Model Tuning	4
2.2	Prompt Design with Hard Prompts	5
2.3	Parameter-Efficient Fine-Tuning	5
3	Prompt Tuning	6
3.1	From Hard Prompts to Soft Prompts	6
3.2	Formal Definition	7
3.3	Parameter and Storage Cost	8
4	Training a Soft Prompt	9
4.1	Initialization	9
4.2	Loss, Gradient, and Update	10
4.3	The Complete Algorithm	11
4.4	What Training Costs	11
5	Worked Example: One Training Step	12
5.1	Task and Setup	12
5.2	Forward Pass and Loss	12
5.3	Backward Pass and Update	13
6	Empirical Evidence	14
6.1	Closing the Gap with Scale	14
6.2	Ablation Study	14
6.3	Robustness to Domain Shift	15
6.4	Prompt Ensembling	16
6.5	Interpretability	17
7	Relation to Other Parameter-Efficient Methods	18
7.1	Where the Trainable Parameters Live	18
7.2	Variants of Prompt Tuning	18
8	Choosing Between Prompt Tuning and Fine-Tuning	20
8.1	Decision Criteria	20
8.2	A Qualitative Comparison	20
9	Challenges and Practical Guidelines	21
9.1	Open Challenges	22
9.2	Guidelines for Writing Task Text	22
10	Discussion	22
10.1	Answers to the Guiding Questions	23
10.2	Limitations of This Study	23
11	Conclusion	24

1. Introduction

Large language models learn a great deal about language during pretraining, but most real applications need them to do one specific job well, such as classifying reviews or answering questions. The usual way to specialize a model is *model tuning*, also called full fine-tuning, which updates every weight of the network on labeled data for the task [1]. Model tuning works well, but it becomes expensive as models grow. A tuned copy of T5-XXL [2] holds 11 billion parameters and takes about 42 GiB of storage, and every new task needs a copy of its own [1].

Prompt design takes the opposite route. The model stays unchanged, and the user describes the task in the input text instead. Brown et al. [3] showed that a frozen GPT-3 can solve new tasks when its prompt contains an instruction and a few examples. Hand-written prompts are fragile, however, and they can hold only as much text as the input allows. Few-shot GPT-3 with 175 billion parameters scores 71.8 on the SuperGLUE benchmark [4], while a fine-tuned T5-XXL scores 89.3 with 16 times fewer parameters [1].

Prompt tuning [1] borrows one idea from each approach. It keeps the model frozen, as prompt design does, and it learns the task from labeled data by gradient descent, as fine-tuning does. We use the following definition throughout the report.

Definition 1: Prompt tuning

Prompt tuning is a parameter-efficient adaptation method. It freezes every weight of a pretrained language model and trains only a short sequence of continuous vectors, the *soft prompt*, which is prepended to the embedded input of every example.

Since the prompt is learned, it can absorb the information in a whole labeled dataset rather than the few examples that fit into a hand-written prompt. Since the model is frozen, one copy of it can serve many tasks, and each task is stored as a small prompt. Lester et al. [1] showed that this simple idea becomes competitive with full fine-tuning once models reach billions of parameters.

1.1. Objectives

Our aim is to explain prompt tuning clearly enough that a reader can follow each step and judge when to use it. Three questions guide the report:

- **Q1.** How does prompt tuning adapt a frozen model, both in theory and during training?
- **Q2.** How does it compare with full fine-tuning and with related methods in cost, accuracy, and robustness?
- **Q3.** When should a practitioner choose it, and which problems are still open?

1.2. Scope and Organization

This report is a literature study, and we did not run any experiments of our own. The method and all measured results come from the paper that introduced prompt tuning [1]. The classification of later methods and the list of open problems come from a 2025 survey by Li, Su, and Collier [5]. We computed the parameter counts and storage figures in Section 3 ourselves from the model sizes reported in [1]. The numbers in the worked example of Section 5 were chosen by hand to keep the arithmetic simple, so they do not come from a real model.

Section 2 reviews the background. Section 3 defines prompt tuning and derives its cost, Section 4 explains how a soft prompt is trained, and Section 5 works through one training step by hand. Section 6 presents the experimental evidence. Section 7 compares prompt tuning with related methods, and Section 8 gives advice on choosing between prompt tuning and fine-tuning. Section 9 covers open challenges and practical guidelines. Section 10 answers the three guiding questions, and Section 11 concludes the report.

2. Background

Prompt tuning grew out of two older ways of adapting a language model. One changes the weights of the model, and the other changes only its input. We review both below, together with the family of parameter-efficient methods to which prompt tuning belongs.

2.1. Language Models and Model Tuning

Modern language models are stacks of Transformer layers [6] that are pretrained on large amounts of unlabeled text. The T5 family [2] treats every task as text generation, including classification. Instead of predicting a class index, T5 writes the label as a word, such as *positive*.

Let $X = (x_1, \dots, x_n)$ be the input tokens, let $Y = (y_1, \dots, y_T)$ be the target tokens, and let $y_{<t}$ denote the target tokens before position t . A model with parameters θ generates the target one token at a time, so that

$$\Pr_{\theta}(Y \mid X) = \prod_{t=1}^T \Pr_{\theta}(y_t \mid y_{<t}, X). \quad (1)$$

Let $\mathcal{D} = \{(X_j, Y_j)\}_{j=1}^{|\mathcal{D}|}$ be a labeled dataset for the task. Model tuning updates all of θ to maximize the likelihood of the targets:

$$\theta^* = \arg \max_{\theta} \sum_{(X,Y) \in \mathcal{D}} \log \Pr_{\theta}(Y \mid X). \quad (2)$$

The trained weights θ^* are specialized to one task. A second task needs a second full copy of the model, and each copy can process only batches from its own task. Figure 1

(left) shows the resulting setup for three tasks.

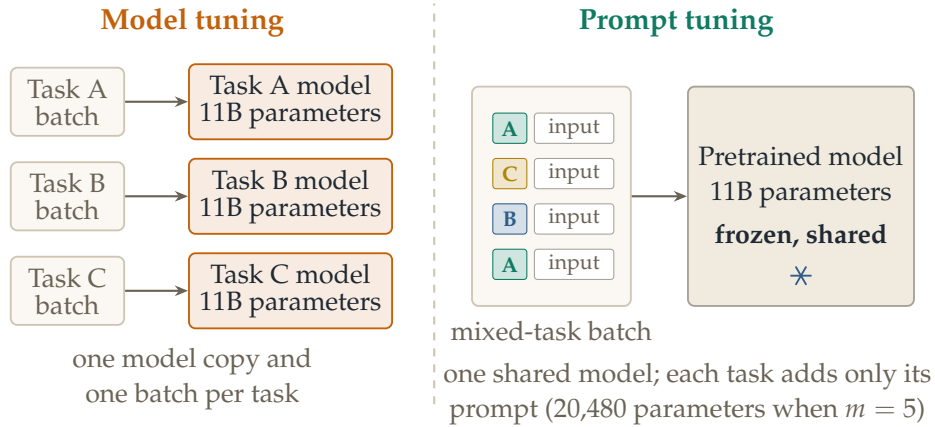


Figure 1: Serving three tasks. Model tuning (left) keeps a separate copy of the whole model for each task and runs a separate batch for each. Prompt tuning (right) keeps one frozen model, and each example carries its own task prompt, so one batch can mix tasks. Adapted from Figure 2 of [1].

2.2. Prompt Design with Hard Prompts

Prompt design leaves θ untouched and adds text to the input instead. Let $[\cdot; \cdot]$ denote the concatenation of two sequences. The model receives a sequence $W = (w_1, \dots, w_m)$ of m vocabulary tokens followed by the input, and it generates Y from $\Pr_{\theta}(Y \mid [W; X])$. The prompt tokens must come from the fixed vocabulary, so finding a good prompt is a discrete search problem. People either write prompts by hand or search the vocabulary for them automatically. Lester et al. [1] point out three weaknesses of this approach. Writing a good task description takes human effort and is easy to get wrong. The prompt competes with the input for the limited input length. The most serious problem is that the quality still falls well short of tuned models, as the SuperGLUE scores in Section 1 show.

2.3. Parameter-Efficient Fine-Tuning

Parameter-efficient methods fall between these two extremes. They train a small set of parameters and leave the rest of the model unchanged.

Definition 2: Parameter-efficient fine-tuning

Parameter-efficient fine-tuning (PEFT) adapts a pretrained model to a task by training a set of parameters that is small compared with the model, while the remaining parameters stay fixed.

Li et al. [5] sort PEFT methods into the four families of Table 1. These methods often come close to full fine-tuning. They can, however, need careful tuning of hyperparameters, and they may lose quality on complex tasks or on small models [5]. The rest of this report focuses on the prompt-based family.

Table 1: Families of parameter-efficient fine-tuning methods, following [5].

Family	What is trained	Example
Addition-based	New modules inserted between Transformer layers	Adapters
Selection-based	A chosen subset of existing weights, such as bias terms	BitFit
Reparameterization-based	Low-rank matrices that represent the weight update	LoRA
Prompt-based	Trainable vectors prepended to the input sequence	Prompt tuning

3. Prompt Tuning

We now define prompt tuning precisely. We first contrast soft prompts with hard prompts, then state the training objective, and finally work out how many parameters and how much storage the method needs.

3.1. From Hard Prompts to Soft Prompts

Prompt tuning drops the requirement that prompt tokens be real words. Each prompt token gets its own vector, which lives in the same space as the word embeddings but is not tied to any word. Because these vectors are continuous, gradient descent can optimize them directly [1]. Table 2 compares the two kinds of prompt, and Figure 2 shows the difference on our running example.

Table 2: Hard prompts and soft prompts compared.

Property	Hard prompt	Soft prompt
Representation	Tokens written by a person	Learnable vectors
Space	Discrete vocabulary	Continuous embedding space
How it is found	Manual writing or discrete search	Gradient descent on labeled data
Use of labeled data	A few in-context examples at most	Every training example
Readable by people	Yes	No
Portable across models	Yes, because the text can be reused	No, because it is tied to the embeddings of one model

Definition 3: Soft prompt

For a model with embedding dimension d , a soft prompt of length m is a trainable matrix $P = [p_1; p_2; \dots; p_m] \in \mathbb{R}^{m \times d}$. Each row p_i is a *virtual token*, a vector in the embedding space that is not tied to any word of the vocabulary.

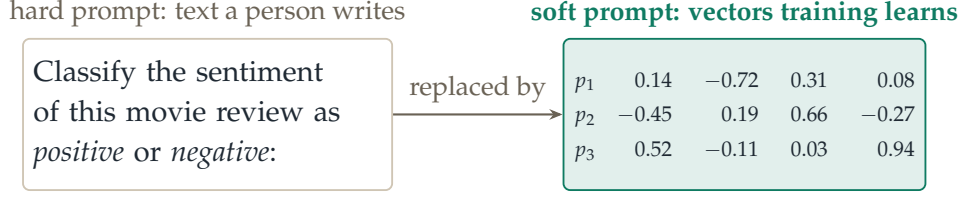


Figure 2: A hard prompt and the soft prompt that replaces it. Each row of the soft prompt is one virtual token. We show four entries per row, whereas a real row has d entries, from 512 for T5-Small to 4096 for T5-XXL. The numbers carry no meaning that a person could read.

3.2. Formal Definition

Table 3 collects the notation used in the rest of the report.

Table 3: Notation.

Symbol	Meaning
$X = (x_1, \dots, x_n)$	input token sequence of length n
$Y = (y_1, \dots, y_T)$	target token sequence, for example a class label written as text
f_θ	pretrained model with parameters θ (frozen during prompt tuning)
d	embedding dimension of the model
$E(\cdot)$	frozen embedding layer, with $E(X) \in \mathbb{R}^{n \times d}$
$P = [p_1; \dots; p_m]$	soft prompt, $P \in \mathbb{R}^{m \times d}$
m	prompt length (number of virtual tokens)
$H = [P; E(X)]$	model input after concatenation, $H \in \mathbb{R}^{(m+n) \times d}$
$\mathcal{L}$	cross-entropy loss
η	learning rate
$\mathcal{D}$	labeled training set
N and K	number of tasks and number of prompts in an ensemble

The frozen embedding layer $E(\cdot)$ turns the input into a matrix $E(X) \in \mathbb{R}^{n \times d}$ with one row per token. Prompt tuning stacks the soft prompt on top of this matrix,

$$H = [P; E(X)] \in \mathbb{R}^{(m+n) \times d}, \quad (3)$$

and feeds H to the pretrained model f_θ as if it were an ordinary embedded sequence of $m + n$ tokens (Figure 3). The model now defines a distribution $\Pr_{\theta;P}(Y \mid [P; E(X)])$ that depends on both the frozen weights θ and the trainable prompt P . Training maximizes the likelihood of the targets over P alone:

$$P^* = \arg \max_P \sum_{(X,Y) \in \mathcal{D}} \log \Pr_{\theta;P}(Y \mid [P; E(X)]) \quad \text{with } \theta \text{ fixed.} \quad (4)$$

Equations (2) and (4) differ only in the variable that is optimized. Model tuning searches over θ , which has billions of entries. Prompt tuning searches over P , which has only $m \cdot d$ entries.

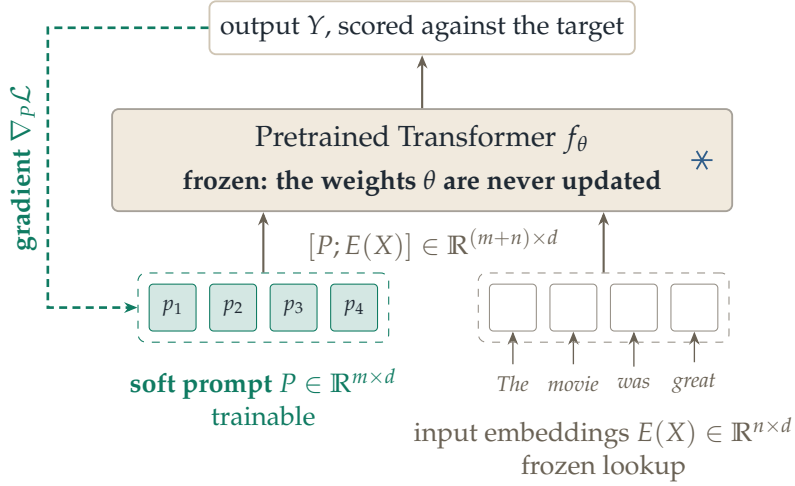


Figure 3: Architecture of prompt tuning. The soft prompt (green) is joined to the embedded input and processed by the frozen Transformer. During training the gradient of the loss (dashed) travels back through the whole network, but only P is updated.

3.3. Parameter and Storage Cost

The number of task-specific parameters is simply the size of P :

$$|P| = m \cdot d. \quad (5)$$

This cost grows with the prompt length and with the width d of the model, but it does not depend on the number of layers. T5-XXL has $d = 4096$, so a five-token prompt has $5 \times 4096 = 20,480$ parameters. The model itself has about 1.1×10^{10} parameters, so the prompt is more than five orders of magnitude smaller [1]. Table 4 gives the exact counts for all five T5.1.1 sizes, and Figure 4 shows how the gap widens with model size. Even at the default length of 100 tokens the prompt stays below 0.07% of the model, and below 0.004% for T5-XXL.

Table 4: Task-specific parameters of a soft prompt for each T5.1.1 size, computed with Equation (5) from the dimensions in Table 4 of [1]. Frozen counts include the embedding table. Each share is relative to the frozen parameters plus the prompt.

Model	d	Frozen parameters	Prompt, $m = 5$		Prompt, $m = 100$	
			Trained	Share	Trained	Share
Small	512	76,961,152	2,560	0.0033%	51,200	0.0665%
Base	768	247,577,856	3,840	0.0016%	76,800	0.0310%
Large	1,024	783,150,080	5,120	0.0007%	102,400	0.0131%
XL	2,048	2,849,757,184	10,240	0.0004%	204,800	0.0072%
XXL	4,096	11,135,332,352	20,480	0.0002%	409,600	0.0037%

The small prompt also lowers the cost of serving many tasks. Let N be the number of tasks, let $|\theta|$ be the number of model parameters, and let $S(N)$ be the number of parameters that must be stored to serve all N tasks. Model tuning stores one model per

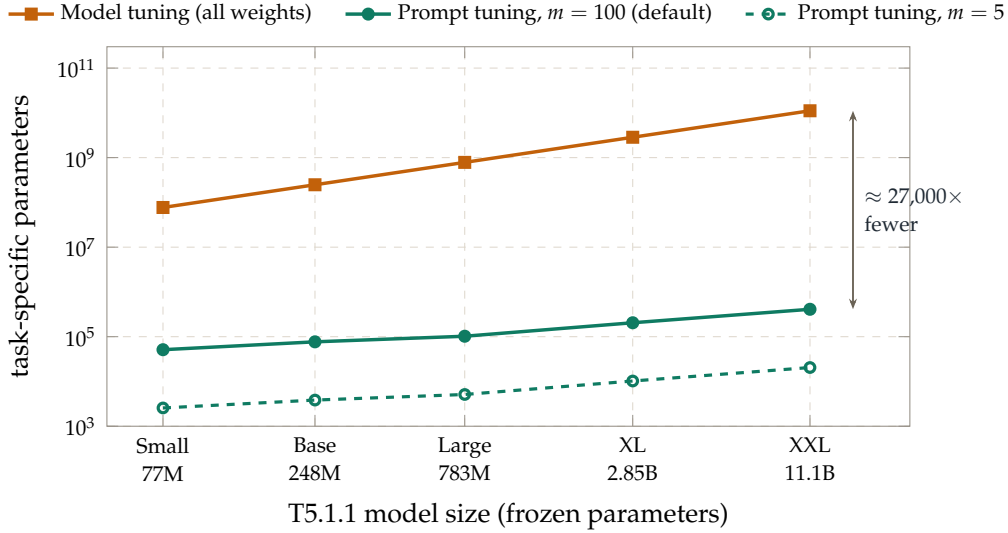


Figure 4: Task-specific parameters against model size on logarithmic axes, using the data of Table 4. At T5-XXL a 100-token prompt is about 27,000 times smaller than a fine-tuned copy of the model.

task, whereas prompt tuning stores one shared model and one prompt per task:

$$S_{\text{model}}(N) = N |\theta| \quad \text{and} \quad S_{\text{prompt}}(N) = |\theta| + N m d. \quad (6)$$

With 32-bit weights one copy of T5-XXL takes about 41.5 GiB (Lester et al. report 42 GiB [1]). A 100-token prompt takes $409,600 \times 4$ bytes, which is about 1.6 MB, and a five-token prompt takes only 80 KiB. Serving 100 tasks therefore needs about 4.05 TiB of storage with model tuning but only about 41.6 GiB with prompt tuning, which is roughly 100 times less. The shared model also allows mixed-task batches, in which every example carries the prompt of its own task (Figure 1, right).

4. Training a Soft Prompt

Training a soft prompt follows the usual supervised learning loop. The one difference is that the optimizer updates a single tensor, the prompt P . Figure 5 shows the whole workflow, and this section explains its steps.

4.1. Initialization

Every row of P needs a starting value before training begins. Lester et al. [1] compared the three strategies listed in Table 5. The last two rest on the same intuition. A soft prompt acts like text placed before the input, so a starting point that looks like real words should help. Class-label initialization goes one step further and points the model toward the valid output labels from the start. For T5-XXL the choice hardly matters, because the differences between the three strategies disappear at that size [1]. Some open-source implementations also let the user start P from the embeddings of a short written task description. The quality of that text then still matters, and we return to this point in Section 9.2.

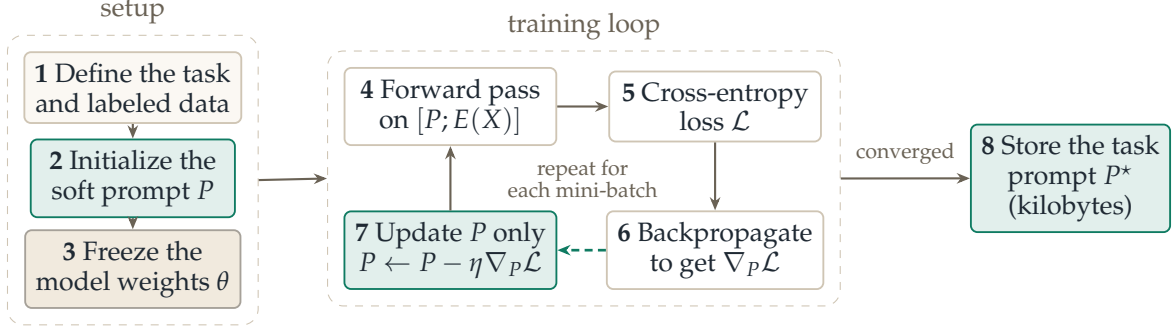


Figure 5: Workflow for training a soft prompt. Steps 1 to 3 form a one-time setup. Steps 4 to 7 then repeat for every mini-batch until the loss converges. Only step 7 changes any parameter, and it changes only P .

Table 5: Initialization strategies for a soft prompt and their effect, as reported in [1].

Strategy	How each row p_i is set	Observed effect
Random uniform	Each entry is drawn uniformly from $[-0.5, 0.5]$.	Worse than the other two strategies for small and mid-sized models.
Sampled vocabulary	Embedding of a token drawn from the 5,000 most common tokens of the vocabulary.	Better than random, because P starts among real word embeddings.
Class label	Embedding of a class label such as <i>positive</i> . Labels with several tokens are averaged, and leftover rows use sampled vocabulary.	Best overall. The label words tend to remain in the learned prompt.

4.2. Loss, Gradient, and Update

For each training pair (X, Y) the model embeds X , prepends P as in Equation (3), and runs a forward pass through the frozen network. We score the prediction with the cross-entropy loss, which is the negative log-likelihood of the target:

$$\mathcal{L}(P) = - \sum_{t=1}^T \log \Pr_{\theta; P}(y_t \mid y_{<t}, [P; E(X)]). \quad (7)$$

Minimizing Equation (7) over the dataset is the same as the maximization in Equation (4). The loss is small when the model gives the correct target a high probability, and it grows without limit as that probability approaches zero.

The prompt enters the network at the very bottom, so the gradient of the loss has to travel back through every layer to reach it. Ordinary backpropagation through f_θ gives the gradient $\nabla_H \mathcal{L}$ with respect to the whole input matrix H . Let $[\cdot]_{1:m}$ denote the first m rows of a matrix. The first m rows of H are the prompt, so the gradient with respect to P is simply

$$\nabla_P \mathcal{L} = [\nabla_H \mathcal{L}]_{1:m}. \quad (8)$$

The weights θ take part in this computation, but they are never changed. With a

learning rate $\eta > 0$, the optimizer then moves P a small step against its gradient:

$$P \leftarrow P - \eta \nabla_P \mathcal{L}. \quad (9)$$

Equation (9) is plain gradient descent. Lester et al. [1] used the Adafactor optimizer, which adapts the step size but otherwise follows the same pattern.

4.3. The Complete Algorithm

Algorithm 1 puts these steps together. Lester et al. trained each prompt for 30,000 steps with a batch size of 32 and a constant learning rate of 0.3, and they chose the final checkpoint by early stopping on the development set [1]. This learning rate is 300 times larger than the rate of 0.001 that the same study used for model tuning.

Algorithm 1 Prompt tuning of a frozen language model

Input: frozen model f_θ ; training set $\mathcal{D}$; prompt length m ; learning rate η ; number of steps S

Output: task-specific soft prompt $P^* \in \mathbb{R}^{m \times d}$

- 1: initialize $P \in \mathbb{R}^{m \times d}$ ▷ Table 5
 - 2: mark every parameter in θ as non-trainable
 - 3: **for** $s = 1$ **to** S **do**
 - 4: sample a mini-batch $\mathcal{B} \subset \mathcal{D}$
 - 5: $\mathcal{L} \leftarrow -\frac{1}{|\mathcal{B}|} \sum_{(X,Y) \in \mathcal{B}} \log \Pr_{\theta;P}(Y \mid [P; E(X)])$ ▷ forward pass, Eq. (7)
 - 6: $G \leftarrow \nabla_P \mathcal{L}$ ▷ backpropagation, Eq. (8)
 - 7: $P \leftarrow P - \eta G$ ▷ only P changes
 - 8: **if** the development score has stopped improving **then**
 - 9: **break** ▷ early stopping
 - 10: **end if**
 - 11: **end for**
 - 12: **return** $P^* \leftarrow P$
-

4.4. What Training Costs

Prompt tuning saves storage and optimizer memory, but training is not free. The optimizer keeps gradients and statistics only for the $m \cdot d$ entries of P , not for the billions of entries of θ . Each step, however, still needs a full forward pass and a full backward pass through the network, because the gradient has to reach the input layer. Every input also grows by m tokens, which adds computation and memory during both training and inference [5]. Soft prompts can also converge slowly, and they are sensitive to the learning rate and to the initialization [5]. On the BoolQ task Lester et al. report training times until convergence that range from a few minutes to almost four hours, depending on the prompt length and the model size [1].

5. Worked Example: One Training Step

Equations (3) to (9) are easier to follow with numbers attached. In this section we trace one training step on a toy sentiment task. We chose the numbers by hand so that the arithmetic is easy to check, and a real model would give different values.

5.1. Task and Setup

The pretrained model already understands English. We want it to label movie reviews as *Positive* or *Negative* without changing any of its weights. Table 6 lists four labeled examples, which are enough to show the procedure. A real task would need a much larger training set for the prompt to converge [5]. For simplicity we treat the two label words as the only outputs that the model can produce.

Table 6: Training examples for the worked example.

Input review	Target label
The movie was fantastic.	Positive
The acting was terrible.	Negative
I really enjoyed this film.	Positive
The story was boring.	Negative

We use a soft prompt of three virtual tokens, $P = [p_1; p_2; p_3]$, and start it from random vectors in the embedding space. The model weights θ stay frozen throughout. The first training example is the review *The movie was fantastic*, whose target label is *Positive*.

5.2. Forward Pass and Loss

We place the prompt in front of the embedded review, and the frozen model produces a probability for each label (Figure 6a). Let q_{pos} and q_{neg} be the probabilities of *Positive* and *Negative*. Suppose that the model assigns

$$q_{\text{pos}} = 0.40 \quad \text{and} \quad q_{\text{neg}} = 0.60.$$

The model prefers *Negative*, so its prediction is wrong. This is expected because the random prompt has not learned anything yet.

The correct class is *Positive*, so Equation (7) reduces to a single term:

$$\mathcal{L} = -\ln q_{\text{pos}} = -\ln 0.40 \approx 0.916. \quad (10)$$

Figure 6b places this value on the curve $\mathcal{L} = -\ln q$. A model that guesses at chance, with $q = 0.5$, would have a loss of $\ln 2 \approx 0.693$. A model that is correct and fully confident would have a loss of 0. On this example our prompt is doing worse than a coin flip.

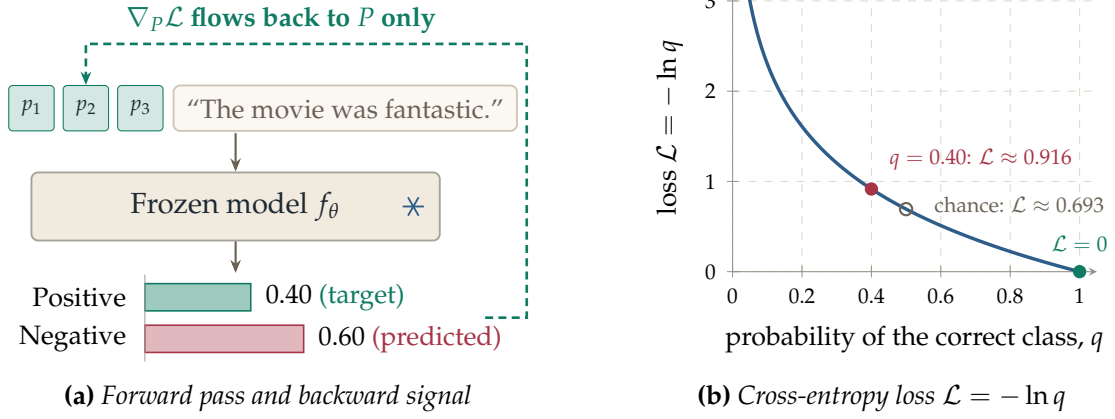


Figure 6: One training step of the worked example. (a) With a random prompt the frozen model gives only 0.40 to the correct label, and the gradient flows back through the model to the prompt. (b) The resulting loss of 0.916 lies above the chance level of 0.693.

5.3. Backward Pass and Update

Let $\mathbf{z} = (z_{\text{pos}}, z_{\text{neg}})$ be the two output scores, or logits, so that $\mathbf{q} = \text{softmax}(\mathbf{z})$, and let $\mathbf{y} = (1, 0)$ be the one-hot target. For softmax followed by cross-entropy, the gradient with respect to the logits has a simple form:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}} = \mathbf{q} - \mathbf{y} = (0.40 - 1, 0.60 - 0) = (-0.60, +0.60). \quad (11)$$

The negative first entry means that raising the score of *Positive* would lower the loss. The positive second entry means that lowering the score of *Negative* would lower it as well.

Backpropagation carries this signal down through every frozen layer to the input matrix H , and Equation (8) keeps the three rows that belong to the prompt. Each vector p_i has d entries, and so does its gradient. To keep the example readable we show one representative entry for each prompt vector:

$$\nabla_P \mathcal{L} \approx [0.20, -0.10, 0.30].$$

A positive entry means that increasing that coordinate would raise the loss, and a negative entry means that increasing it would lower the loss. With a learning rate of $\eta = 0.1$, Equation (9) gives

$$P_{\text{new}} = P_{\text{old}} - \eta \nabla_P \mathcal{L} = P_{\text{old}} - 0.1 \begin{bmatrix} 0.20 \\ -0.10 \\ 0.30 \end{bmatrix} = P_{\text{old}} + \begin{bmatrix} -0.02 \\ +0.01 \\ -0.03 \end{bmatrix}. \quad (12)$$

The minus sign moves P toward a lower loss. Only the three prompt vectors change, and every weight of the model stays exactly as it was.

Training then moves on to the next example and repeats the same cycle of forward pass, loss, gradient, and update. In the beginning the prompt holds random vectors, so

the predictions are poor and the loss is high. Over thousands of steps the updates shape these vectors into a task-specific representation that steers the frozen model toward the correct label. Once training converges, we save the final prompt P^* . It is the only thing that has to be stored for this task.

6. Empirical Evidence

This section summarizes the experiments of Lester et al. [1], who introduced prompt tuning. All numbers in this section are quoted from their paper. They trained prompts on frozen T5.1.1 models of five sizes, from Small with 77 million parameters to XXL with 11 billion, and they evaluated the prompts on SuperGLUE, a benchmark of eight English language-understanding tasks [4]. Every prompt covered a single task, and each configuration was repeated three times. Table 7 lists the remaining settings.

Table 7: *Main settings in the experiments of Lester et al. [1].*

Setting	Value
Default prompt	100 tokens with class-label initialization
Model preparation	T5 adapted with a language-modeling objective for 100K extra steps (Section 6.2)
Optimizer	Adafactor with a constant learning rate of 0.3 and a batch size of 32
Training length	30,000 steps, with the checkpoint chosen by early stopping
Baselines	Model tuning per task, multi-task model tuning, and few-shot prompt design with GPT-3

6.1. Closing the Gap with Scale

The main result is that prompt tuning becomes more competitive as the model grows. On small models it trails model tuning by a wide margin. Prompt tuning on the XXL model matches even the stronger multi-task model-tuning baseline, although it uses over 20,000 times fewer task-specific parameters [1]. The survey by Li et al. confirms that at this scale prompt tuning comes within a couple of percentage points of full tuning on most SuperGLUE tasks [5].

Prompt tuning also clearly beats prompt design. A prompt-tuned T5-Small matches GPT-3 XL, a model more than 16 times larger. A prompt-tuned T5-Large beats the full GPT-3 with 175 billion parameters, which is more than 220 times larger [1]. Lester et al. explain this with the amount of data that each kind of prompt can use. A learned prompt absorbs the signal from every labeled example, whereas a hand-written prompt can hold only a few examples.

6.2. Ablation Study

Lester et al. varied one design choice at a time and kept the others at their default values. Table 8 summarizes the four ablations. They all show the same pattern. The

largest model tolerates almost every suboptimal choice.

Table 8: Ablation results reported by Lester et al. [1]. The last column shows how the largest model behaves.

Design choice	Values tested	Main finding	At XXL size
Prompt length m	1, 5, 20, 100, 150	Going beyond one token is critical for most sizes. Gains beyond 20 tokens are small, and more than 100 tokens is slightly harmful.	Strong even with a one-token prompt
Initialization	random uniform, sampled vocabulary, class label	Class-label initialization is best, and random initialization is worst.	Differences vanish
Pre-training objective	span corruption, span corruption with sentinel, LM adaptation	LM adaptation is best. Models trained only with span corruption are unreliable.	Works with any objective
LM adaptation length	0, 10K, 50K, 100K steps	Longer adaptation helps, up to 100K steps.	Robust even to short adaptation

The pre-training objective needs a short explanation. The standard T5 model is pre-trained with *span corruption*. In this task the model rebuilds spans of text that were masked out and marked with special sentinel tokens [1]. Such a model has never seen natural text without sentinels, and a prompt alone cannot easily undo this habit, because the frozen decoder cannot change. In this setting the mid-sized models often never learned to output a valid class label and scored 0%, and only two of the five model sizes worked well. Lester et al. solved the problem with *LM adaptation*. They continued training T5 for a small number of extra steps with a language-modeling objective, in which the model continues a natural piece of text. They performed this adaptation only once and then reused the frozen model for every task.

6.3. Robustness to Domain Shift

Freezing the model also appears to make it more robust. Prompt tuning cannot change what the model knows about language in general. It can only influence how the input is represented. Lester et al. argue that this limits the ability of the model to overfit a dataset by memorizing surface cues [1]. They tested this idea with zero-shot domain transfer, in which a model is trained on one dataset and then evaluated, without further training, on a dataset from a different domain.

Table 9 shows the results for question answering. Lester et al. trained both methods on SQuAD, whose passages come from Wikipedia, and evaluated them on out-of-domain datasets from the MRQA 2019 shared task [1]. Prompt tuning wins on four of the six datasets. Its largest gain of 12.5 F1 points appears on TextbookQA, one of the

datasets that differ most from Wikipedia. Model tuning comes out ahead on DuoRC and on DROP, which shares its Wikipedia domain with SQuAD.

Table 9: Zero-shot domain transfer for question answering. Entries are F1 scores (mean $\pm$ standard deviation of three runs) for models trained on SQuAD, and the column Δ is prompt tuning minus model tuning. From Table 1 of [1].

Dataset	Domain	Model tuning	Prompt tuning	Δ
SQuAD (in-domain)	Wikipedia	94.9 $\pm$ 0.2	94.8 $\pm$ 0.1	−0.1
TextbookQA	Book	54.3 $\pm$ 3.7	66.8 $\pm$ 2.9	+12.5
BioASQ	Biomedical	77.9 $\pm$ 0.4	79.1 $\pm$ 0.3	+1.2
RACE	Exam	59.8 $\pm$ 0.6	60.7 $\pm$ 0.5	+0.9
RE	Wikipedia	88.4 $\pm$ 0.1	88.8 $\pm$ 0.2	+0.4
DuoRC	Movie	68.9 $\pm$ 0.7	67.7 $\pm$ 1.1	−1.2
DROP	Wikipedia	68.9 $\pm$ 1.7	67.1 $\pm$ 1.9	−1.8

The second test uses two paraphrase-detection tasks. QQP asks whether two questions from Quora are duplicates, and MRPC asks whether two sentences from news articles are paraphrases. Table 10 shows that a prompt trained on QQP transfers to MRPC much better than a fully tuned model, by 3.2 accuracy points and 3.1 F1 points. In the other direction the two methods are close. Lester et al. conclude that model tuning may be over-parameterized and therefore more likely to overfit its training task [1].

Table 10: Zero-shot transfer between two paraphrase-detection tasks (mean $\pm$ standard deviation). From Table 2 of [1].

Train	Evaluate	Tuning	Accuracy	F1
QQP	MRPC	Model	73.1 $\pm$ 0.9	81.2 $\pm$ 2.1
		Prompt	76.3 $\pm$ 0.1	84.3 $\pm$ 0.3
MRPC	QQP	Model	74.9 $\pm$ 1.3	70.9 $\pm$ 1.2
		Prompt	75.4 $\pm$ 0.8	69.7 $\pm$ 0.3

6.4. Prompt Ensembling

Ensembles of models trained from different starting points usually beat a single model, but ensembles of large models are expensive. An ensemble of K copies of T5-XXL needs about $K \times 42$ GiB of storage and K separate forward passes. Prompt tuning makes ensembles much cheaper. The ensemble consists of K prompts $P_1, \dots, P_K$ trained for the same task on one shared model. To classify an input X , we copy it K times into a single batch, attach a different prompt to each copy, and combine the K predictions by

majority vote into the final prediction $\hat{Y}$:

$$\hat{Y} = \text{mode} \left\{ \arg \max_Y \Pr_{\theta; P_k} (Y \mid [P_k; E(X)]) \right\}_{k=1}^K. \quad (13)$$

Lester et al. trained five prompts for each SuperGLUE task on one frozen T5-XXL [1]. Table 11 shows that the ensemble beats the average prompt on every task and matches or beats the best single prompt. The overall SuperGLUE score rises from 90.5 for the average prompt to 91.3 for the ensemble.

Table 11: Five-prompt ensemble on a single frozen T5-XXL model (SuperGLUE development set). From Table 3 of [1].

Dataset	Metric	Average prompt	Best prompt	Ensemble
BoolQ	accuracy	91.1	91.3	91.7
CB	accuracy/F1	99.3 / 99.0	100.0 / 100.0	100.0 / 100.0
COPA	accuracy	98.8	100.0	100.0
MultiRC	EM/F1 _a	65.7 / 88.7	66.3 / 89.0	67.1 / 89.4
ReCoRD	EM/F1	92.7 / 93.4	92.9 / 93.5	93.2 / 93.9
RTE	accuracy	92.6	93.5	93.5
WiC	accuracy	76.2	76.6	77.4
WSC	accuracy	95.8	96.2	96.2
SuperGLUE	average	90.5	91.0	91.3

6.5. Interpretability

Soft prompts live in a continuous space, so people cannot read them directly. To inspect them Lester et al. looked up the vocabulary tokens nearest to each learned prompt vector [1]. For a prompt vector p_i and the embedding v of a vocabulary token, they measured closeness with the cosine similarity

$$\text{sim}(p_i, v) = \frac{\langle p_i, v \rangle}{\|p_i\| \|v\|}. \quad (14)$$

The five nearest neighbors of a prompt vector often form a tight group of related words, such as *Technology*, *technology*, *Technologies*, *technological*, and *technologies*. Random vectors show no such grouping, which suggests that the prompts learn word-like representations. With class-label initialization the label words often remain among the neighbors after training, so the prompt seems to keep the expected output classes as a reference. For BoolQ the words *science*, *technology*, and *engineering* appear often, and about 20% of BoolQ questions belong to the Nature/Science category. The prompt may therefore prime the model to read inputs in a particular domain. Taken as whole sequences, however, the learned prompts cannot be interpreted.

7. Relation to Other Parameter-Efficient Methods

Prompt tuning is one of several methods that adapt a frozen model. In this section we compare it with its closest relatives and then give an overview of the variants that researchers have built on it since 2021.

7.1. Where the Trainable Parameters Live

The methods differ mainly in where they put the trainable parameters, as Figure 7 shows. Adapters [7] insert small bottleneck layers between the frozen layers, so they change the function that the network computes. Prompt tuning leaves that function alone and adds new inputs instead, which influence how the rest of the input is processed [1]. Prefix tuning [8] lies between the two. It learns prefix activations at every Transformer layer, and these activations are the same for every example. Prompt tuning adds a single prompt at the input and lets the network compute the deeper representations in the context of each example. Table 12 extends the comparison to three further methods.

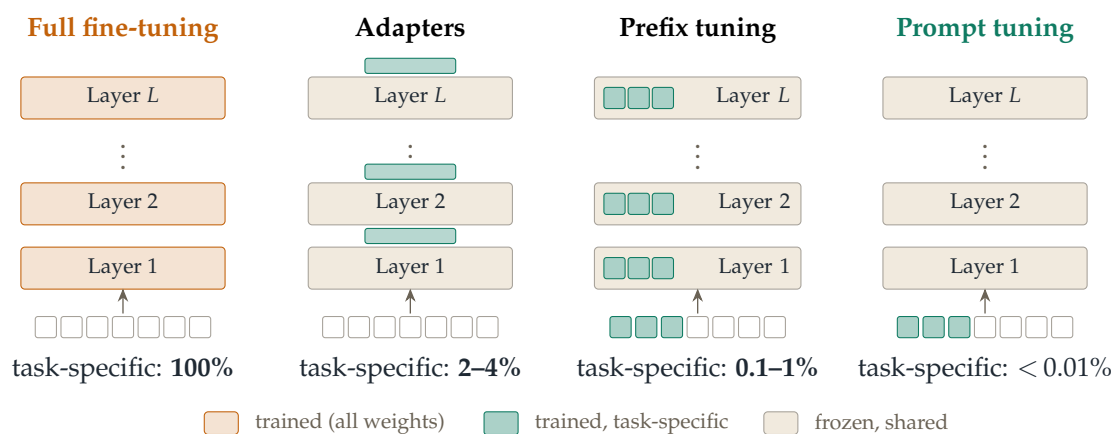


Figure 7: Where each method places its trainable parameters. The shares of task-specific parameters are those reported in [1] for T5.1.1 and, for adapters, in [7].

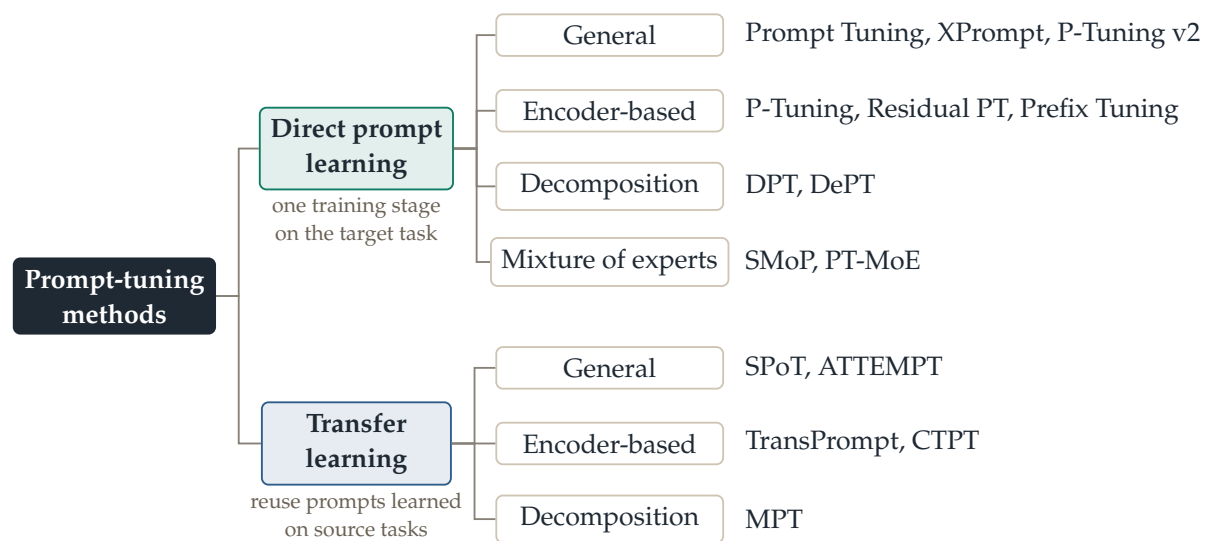
7.2. Variants of Prompt Tuning

Many variants have extended the original method. Li et al. [5] divide them into the two branches of Figure 8. Direct prompt learning methods train a soft prompt in a single stage on the target task. Transfer learning methods first learn prompts on source tasks with plenty of labeled data and then reuse them on target tasks that have little data.

Several direct methods change how the prompt is represented or where it is placed. XPrompt removes prompt tokens that do not help, and P-Tuning v2 adds soft prompts to every layer instead of only to the input [5]. Decomposed prompt tuning (DPT) [9] builds on the observation that soft prompts tend to become low-rank during training. It therefore chooses a small rank r with $r \ll \min(m, d)$ and writes the prompt as the

Table 12: Prompt tuning compared with related adaptation methods, after Section 4 and Figure 4 of [1].

Method	Trained parameters	Task-specific share	Remarks
Model tuning	All weights	100%	One model copy per task.
Adapters	Bottleneck layers between frozen layers	2–4%	Changes the function of each layer.
Prefix tuning	Prefix activations at every layer	0.1–1%	Needs extra parameters during training for stability.
WARP	Input and output layers of a masked language model	< 0.1%	Limited to classification.
P-tuning	Continuous prompts mixed into the input	–	Needs joint model tuning for strong SuperGLUE results.
Prompt tuning	Soft prompt at the input only	< 0.01%	Model unchanged, and mixed-task batches are possible.
Prompt design	None, only 500 to 2,000 prompt tokens	–	No training, but the lowest task quality.

**Figure 8:** Taxonomy of prompt-tuning methods, following Figure 1 of [5].

product of two thin matrices:

$$P = AB \quad \text{with} \quad A \in \mathbb{R}^{m \times r} \text{ and } B \in \mathbb{R}^{r \times d}. \quad (15)$$

This form needs only $r(m + d)$ parameters instead of $m \cdot d$. Mixture-of-experts methods such as SMoP keep several short prompts and let a small router choose one for each input [5].

Transfer methods reuse what prompts have learned on other tasks. SPoT starts the target prompt from a prompt trained on one or more source tasks [5]. ATTEMPT [10] trains prompts $P_1, \dots, P_t$ on t source tasks and adds a trainable prompt P_{target} for the

target task, which also serves as P_{t+1} . For each input of the target task it computes attention weights $a_1, \dots, a_{t+1}$ and mixes all $t + 1$ prompts:

$$P_{\text{input}} = P_{\text{target}} + \sum_{j=1}^{t+1} a_j P_j. \quad (16)$$

Multitask prompt tuning (MPT) [11] shares one prompt matrix $P_{\text{sh}} \in \mathbb{R}^{m \times d}$ across tasks and gives each task k only a rank-one correction. Let $\circ$ denote the element-wise product of two matrices. The prompt of task k is then

$$\hat{P}_k = P_{\text{sh}} \circ (u_k v_k^\top) \quad \text{with} \quad u_k \in \mathbb{R}^m \text{ and } v_k \in \mathbb{R}^d. \quad (17)$$

These variants generally keep the language model frozen. The limitations listed in the survey point to a common trade-off. Most variants gain stability or accuracy at the cost of extra parameters, extra hyperparameters, or a more complex training procedure [5].

8. Choosing Between Prompt Tuning and Fine-Tuning

Neither prompt tuning nor full fine-tuning is the best choice in every situation. Figure 9 places both on a spectrum together with prompt engineering. Prompt engineering is the cheapest option, but its behavior depends on the exact wording of each prompt. Full fine-tuning gives the most control, but it costs a full model copy per task. Prompt tuning lies between them, because it learns consistent behavior from data without updating every weight.

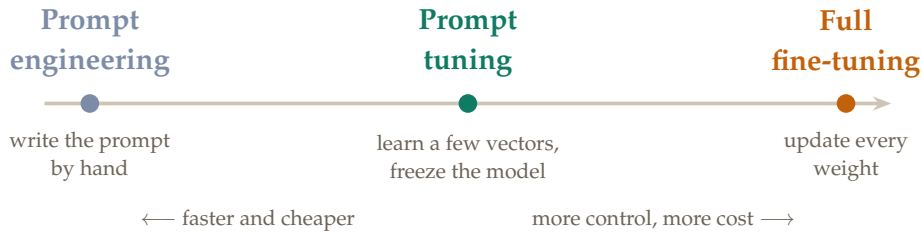


Figure 9: The spectrum of adaptation methods, from writing the prompt by hand to updating every weight.

8.1. Decision Criteria

Table 13 turns the evidence of Sections 3 to 7 into practical criteria. No single criterion settles the choice. Prompt tuning suits a large model that must serve many tasks and stay unchanged. Full fine-tuning remains the safer choice for small models and for tasks that require genuinely new behavior or knowledge.

8.2. A Qualitative Comparison

Figure 10 repeats the comparison from our presentation. It rates both methods on five axes, namely the speed of putting the method to work, the amount of labeled data

Table 13: *Criteria for choosing between prompt tuning and full fine-tuning.*

Criterion	Favors prompt tuning	Favors full fine-tuning
Model size	Billions of parameters, where the quality gap closes	Small or mid-sized models, where prompt tuning lags
Number of tasks	Many tasks served from one shared model	One task or very few
Base model	Must stay unchanged, for example to keep its general abilities	May be modified freely
Storage	Kilobytes to megabytes per task	A full model copy per task is affordable
Nature of the task	Steering abilities that the model already has	Deep changes in behavior or new domain knowledge
Labeled data	Enough to train a prompt, but not necessarily a large corpus	Large labeled datasets
Deployment domain	Inputs may drift away from the training domain	Inputs match the training domain closely

needed, the quality of the output, the amount of change to the model, and the flexibility of reuse across tasks. The chart is our own qualitative judgment and not a measurement. It reflects typical small and mid-sized models. At the scale of T5-XXL the gap in output quality largely closes (Section 6.1).

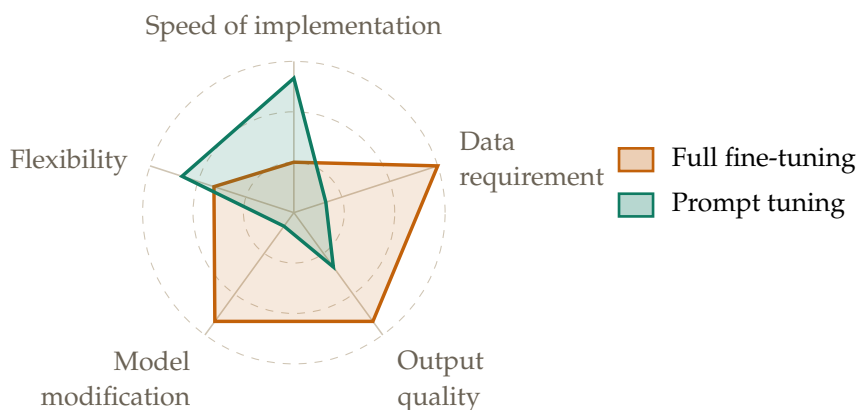


Figure 10: *Qualitative comparison of full fine-tuning and prompt tuning. The ratings on a scale of 1 to 9 are the ones used in our presentation. They are judgments, not measurements.*

9. Challenges and Practical Guidelines

Prompt tuning is still an active research area. This section lists the problems that remain open and then gives guidelines for the one part of the pipeline that people still write by hand, the task text.

9.1. Open Challenges

Li et al. [5] name five challenges that current prompt-tuning methods share, together with directions for future work. Table 14 places each challenge next to the direction that addresses it most directly. The pairing is ours.

Table 14: *Open challenges of prompt tuning and directions for future work, following Section 5 of [5].*

Challenge	Description	Direction
Computational efficiency	Soft prompts lengthen every input, which raises memory use and computation.	Lightweight or dynamically generated prompts, and prompts shared across tasks.
Training stability	Results depend strongly on hyperparameters, especially the learning rate, and convergence can be slow.	More robust optimization and adaptive learning strategies.
Initialization	Random and embedding-based starting points can both lead to weak prompts.	Better initialization strategies.
Model-scale dependency	The method works well on large models but degrades on small ones.	Transfer from source tasks and meta-learning for prompt adaptation.
Explainability	The meaning of learned prompts, and how they interact with the model, is poorly understood.	Analysis of optimization paths and dedicated explainability methods.

9.2. Guidelines for Writing Task Text

Soft prompts are learned, but text written by people still enters the pipeline in two places. It appears when a prompt is initialized from class labels or from a task description (Section 4.1), and it appears when a hand-written prompt serves as a baseline. Our presentation collected ten guidelines for writing such text. Table 15 groups them into six.

These guidelines are close to the rules of technical writing taught in this course. They ask the writer to say exactly what is meant, to avoid ambiguity, and to prefer positive statements. We found this link helpful while preparing the presentation. A prompt, like a report, has to make sense to a reader who cannot ask the writer what was meant.

10. Discussion

We now return to the three guiding questions of Section 1.1 and then state the limits of what this study can claim.

Table 15: *Guidelines for writing clear task text for a language model.*

Guideline	What it means in practice
State the core request first	Put the main question or action at the start, and keep only the details that serve it.
Break the request down	Split a request with several parts into focused pieces, and separate them with bullet points or numbered lists.
Use clear, positive language	Avoid jargon, ambiguous terms, and double negatives, and state what is required.
Give only the context needed	Leave out background that does not serve the request, but include what the model genuinely needs.
Specify constraints	Name any topics or references to avoid explicitly.
Test for clarity	Ask someone else to read the prompt, and split long requests into separate prompts.

10.1. Answers to the Guiding Questions

Q1. Prompt tuning learns a matrix $P \in \mathbb{R}^{m \times d}$ that is prepended to the embedded input (Equation (3)). Training uses ordinary backpropagation, but only the rows of the gradient that belong to P are used, and only P is updated (Equations (8) and (9)). The worked example of Section 5 shows one such step, in which no model weight changes.

Q2. Among methods that learn parameters, prompt tuning needs the fewest. For models with more than one billion parameters its task-specific share is below 0.01%. At 11 billion parameters it matches full fine-tuning on SuperGLUE, and it clearly outperforms hand-written GPT-3 prompts, even when the GPT-3 model is far larger. It also transfers better to new domains and makes ensembles and multi-task serving cheap. Its weak points are lower quality on small models and a training loop that still needs full forward and backward passes.

Q3. Prompt tuning is the better choice for a large model that must serve many tasks, stay unchanged, and fit a small storage budget. Full fine-tuning remains preferable for small models, for deep changes in behavior, and when the highest accuracy on a single task matters more than cost. Training stability, initialization, dependence on model size, and interpretability are still open problems.

10.2. Limitations of This Study

This study has four limitations. It is a literature study, and nearly all of its measured results come from one paper [1], which used T5 models and English benchmarks. The numbers in the worked example are illustrative rather than measured. The radar chart in Figure 10 is a qualitative judgment. We also did not compare prompt tuning directly with other popular PEFT methods such as LoRA [12]. Such a comparison would need controlled experiments on the same models and tasks.

11. Conclusion

In this report we studied prompt tuning, which adapts a frozen language model by learning a short soft prompt instead of changing the weights of the model. We expressed the method as the optimization of a matrix $P \in \mathbb{R}^{m \times d}$ that is placed before the input embeddings. For prompts of up to 100 tokens its cost of $m \cdot d$ parameters per task is below 0.004% of T5-XXL. By following one training step by hand we showed why the gradient changes only the prompt. The experiments of Lester et al. show that prompt tuning closes the gap with full fine-tuning as models grow to billions of parameters. They also show that it holds up better under domain shift and that it makes ensembles and multi-task serving inexpensive.

Prompt tuning does not replace fine-tuning in every case. It is weaker on small models, its training can be slow and sensitive to hyperparameters, and people cannot read its prompts. We still find its main idea convincing. When the parameters that define a task are kept apart from the parameters that hold general knowledge of language, one frozen model can serve many tasks, and each extra task costs only a few kilobytes.

References

- [1] B. Lester, R. Al-Rfou, and N. Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3045–3059, 2021. arXiv:2104.08691.
- [2] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.
- [3] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.
- [4] A. Wang, Y. Pruksachatkun, N. Nangia, A. Singh, J. Michael, F. Hill, O. Levy, and S. Bowman. SuperGLUE: A stickier benchmark for general-purpose language understanding systems. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [5] Z. Li, Y. Su, and N. Collier. A survey on prompt tuning. In *3rd Workshop on Efficient Systems for Foundation Models at the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2507.06085.
- [6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.
- [7] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, PMLR 97, pages 2790–2799, 2019.
- [8] X. L. Li and P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of ACL-IJCNLP 2021*, pages 4582–4597, 2021.
- [9] Y. Xiao, L. Xu, J. Li, W. Lu, and X. Li. Decomposed prompt tuning via low-rank reparameterization. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 13335–13347, 2023.
- [10] A. Asai, M. Salehi, M. Peters, and H. Hajishirzi. ATTEMPT: Parameter-efficient multi-task tuning via attentional mixtures of soft prompts. In *Proceedings of EMNLP 2022*, pages 6655–6672, 2022.
- [11] Z. Wang, R. Panda, L. Karlinsky, R. Feris, H. Sun, and Y. Kim. Multitask prompt tuning enables parameter-efficient transfer learning. In *International Conference on Learning Representations (ICLR)*, 2023.
- [12] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.